

Topic Modeling

Non-negative Matrix Factorization (NMF)

Non-negative Matrix Factorization (NMF)

- a technique used in linear algebra and machine learning for reducing the dimensionality of non-negative data
- and uncovering latent (hidden) patterns.

It's called "non-negative" because it imposes the constraint that all elements in the matrices involved must be non-negative (i.e., greater than or equal to zero). This constraint makes NMF particularly suitable for data that naturally contains only non-negative values, such as images, text, and audio spectra.

Non-negative Matrix Factorization (NMF)

The factorization aspect of NMF refers to the process of decomposing a non-negative matrix V into two non-negative matrices W and H , such that: $V \approx WH$

- V is an $m \times n$ matrix representing the original data,
- W is an $m \times k$ matrix representing the basis vectors or features,
- H is a $k \times n$ matrix representing the coefficients or weights.

The value k is a hyperparameter that determines the reduced dimensionality, and it's typically chosen to be smaller than m and n .

Mathematical Formulation

The goal of NMF is to find the matrices W and H that minimize the reconstruction error between V and WH . This is often formulated as an optimization problem, where the objective is to minimize a cost function such as the Frobenius norm:

$$\min_{W,H} ||V - WH||_F^2$$

subject to $W \geq 0$ and $H \geq 0$, where $||\cdot||_F$ denotes the Frobenius norm, and the inequalities are element-wise.

Frobenius Form

NMF is a kind of unsupervised machine learning technique. The main goal of unsupervised learning is to quantify the distance between the elements.

“Frobenius Form” is defined by the square root of the sum of absolute squares of its elements. It is also known as the Euclidean norm. The formula for calculating the Frobenius Norm is given by:

$$\|A\|_F \equiv \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

Algorithms

- 1. Multiplicative Update Rules:** This is one of the most popular algorithms for NMF, introduced by Lee and Seung. It uses iterative update rules that multiplicatively adjust the matrices W and H to minimize the reconstruction error. It's simple to implement and has been widely used in practice. However, it can sometimes be **slow to converge**.
- 2. Alternating Least Squares (ALS):** In ALS, the optimization problem is divided into two subproblems, one for W and one for H , and solved alternately. This method can be faster than multiplicative updates, especially when the matrices are large, but it **might not handle sparsity** as well.
- 3. Projected Gradient Methods:** These methods use gradient descent to minimize the objective function, with projection steps to ensure that the matrices W and H remain non-negative. They can be faster than multiplicative update rules and are **suitable for large-scale problems**.
- 4. Non-smooth NMF (nsNMF):** This variant introduces a sparseness constraint on the matrices W and H , which can be useful for applications where **interpretability is important**, such as in text mining or image analysis.
- 5. Convex-NMF:** This approach reformulates the NMF problem as a convex optimization problem, **which can lead to more robust solutions**. It's particularly useful when the data has a **clear convex structure**.

Algorithms Selection Criteria

- **Data Size:** For large datasets, algorithms like ALS or projected gradient methods might be more efficient than multiplicative update rules.
- **Sparsity:** If the data matrix V is sparse, algorithms that can handle sparsity, such as ALS or nsNMF, might be more appropriate.
- **Interpretability:** If interpretability of the factors is important, algorithms like nsNMF that enforce sparsity constraints can be beneficial.
- **Convergence Speed:** Some algorithms converge faster than others, so if computational time is a concern, you might prefer faster algorithms like ALS or projected gradient methods.
- **Robustness:** In cases where the data has noise or outliers, robust algorithms like Convex-NMF might be more suitable.

Solvers

- 1.'cd' - Coordinate Descent:** This solver iteratively optimizes the objective function with respect to one variable at a time while keeping others fixed. In the context of NMF, it alternates between updating the components of W and H , making it efficient for problems with a large number of features.
- 2.'mu' - Multiplicative Update:** A popular choice for NMF, this algorithm updates W and H by multiplying their current values by ratios derived from the data and the current approximation. It's known for its simplicity and the fact that it naturally enforces non-negativity without needing additional projections.

Multiplicative Update Rules

One of the most popular algorithms for NMF is based on multiplicative update rules proposed by Lee and Seung. The update rules for W and H are as follows:

$$\begin{aligned} W &\leftarrow W \odot \frac{VH^T}{WHH^T} \\ H &\leftarrow H \odot \frac{W^TV}{W^TWH} \end{aligned}$$

Here, $\odot$ denotes element-wise multiplication, and the division is also element-wise. These rules are applied iteratively until convergence. The multiplicative nature of the updates ensures that W and H remain non-negative throughout the iterations.

Alternating Least Squares (ALS)

In the ALS approach, the update of W and H is done in an alternating fashion:

- 1.Update W :** Fix H and solve for W by minimizing the objective function with respect to W . This can be done using least squares optimization, often with additional constraints to enforce non-negativity.
- 2.Update H :** Fix W and solve for H by minimizing the objective function with respect to H , again using least squares optimization with non-negativity constraints.

The updates for W and H in ALS are not multiplicative but involve solving optimization problems, typically using quadratic programming or other optimization techniques that can handle non-negativity constraints.

Hyperparameters

- **Number of Components (Rank k):**
 - This is perhaps the most critical hyperparameter. It determines the number of latent features and affects the granularity of the decomposition.
 - **Tuning Strategy:** Use domain knowledge or consider the data's intrinsic dimensionality. Cross-validation can help determine the rank that gives the best trade-off between reconstruction error and model complexity.
- **Solver:**
 - Different solvers can be used to optimize the factorization, such as 'cd' (coordinate descent) or 'mu' (multiplicative update).
 - **Tuning Strategy:** Choose 'cd' for faster convergence on dense data, and 'mu' can be better for sparse data or when a more precise convergence is needed.
- **Beta Loss:**
 - The beta loss function to be minimized, measuring the difference between the original matrix V and its approximation WH . It can be 'frobenius' (Frobenius norm), 'kullback-leibler' (KL divergence), or 'itakura-saito' (Itakura-Saito divergence).
 - **Tuning Strategy:** Choose the loss function based on the distribution of your data. For example, KL divergence can be suitable for count data, while the Frobenius norm is often used for Euclidean distances.

Hyperparameters

- **Max Iterations:**
 - The maximum number of iterations to compute.
 - **Tuning Strategy:** Set a reasonable default based on computational budget. If the algorithm converges early, the exact value might not be critical.
- **Initialization Method:**
 - How the initial values for W and H are chosen ('random', 'nndsvd', 'nndsvda', 'nndsvdar', or custom).
 - **Tuning Strategy:** 'nndsvd' (Nonnegative Double Singular Value Decomposition) is a good default that often leads to faster convergence.
- **Tolerance:**
 - The stopping condition for the iterative process when the improvement of the reconstruction error is below a certain threshold.
 - **Tuning Strategy:** Set a default tolerance and adjust it if the model is not converging or is taking too long to do so.
- **Regularization:**
 - $L1$ and $L2$ regularization terms for W and H , which can help prevent overfitting and improve generalization.
 - **Tuning Strategy:** Use cross-validation to find a good balance between regularization and reconstruction error. Start with no regularization and increase it if the model overfits.

Hyperparameters

- **Alpha:**
 - The constant that multiplies the regularization terms, if regularization is used.
 - **Tuning Strategy:** Adjust this in conjunction with the regularization terms during cross-validation.
- **Random State:**
 - The seed of the pseudo-random number generator to use when shuffling the data and initializing the factors.
 - **Tuning Strategy:** Keep it fixed if you want reproducible results. Vary it if you suspect the initialization is affecting the solution.
- **Shuffle:**
 - Whether to shuffle the samples before each iteration.
 - **Tuning Strategy:** Shuffling might help in escaping local minima but might also increase computation time. It's usually not a critical parameter to tune.

Initialization methods

- **Random Initialization:**

- Matrices W and H are initialized with random non-negative values.
- This method is straightforward but can lead to longer convergence times and a higher chance of finding suboptimal local minima.

- **Nonnegative Double Singular Value Decomposition (NDSVD):**

- This method is based on the Singular Value Decomposition (SVD). It is designed to provide a good starting point for NMF by avoiding the potential for slow convergence associated with random initialization.
- NDSVD decomposes the original non-negative matrix V into W and H matrices using SVD, and then it makes necessary adjustments to ensure non-negativity.
- It's especially useful when the data matrix V is sparse.

Initialization methods

- **NNDSVDA:**

- It is a variant of NNDSVD. After applying SVD, the negative elements of the initialized matrices are set to the mean value of the positive elements.
- This method provides an averaging effect that can lead to better convergence properties in practice.

- **NNDSVDar:**

- This method is similar to NNDSVDA but adds random non-negative values to the factors W and H .
- The addition of random values can help in achieving more diverse solutions and possibly avoiding local minima.

Initialization methods

- **Custom Initialization:**

- If domain knowledge suggests a certain structure or pattern in the data, custom initialization methods can be developed to incorporate this knowledge directly into the initialization of W and H .
- For example, if the data is known to have a particular sparsity pattern or block structure, the initialization can be designed to reflect that structure.